SISTEMA KANBAN

Universidade de Brasília — Instituto de Ciências Exatas — Departamento de Ciência da Computação

Técnicas de Programação 1 — Trabalho Prático — Semestre 2026.2

MODELO DE ARQUITETURA DE SOFTWARE

Arthur Vieira **Gabriel Sales** **Isabela Honda Ferreira** **João Roberto Machado**

1. Visão Geral e Propósito do Sistema

O sistema de software tem o propósito de prover suporte à gestão de projetos pelo método Kanban. Em cada projeto gerenciado, os fluxos de trabalho são organizados em quadros compostos por três colunas estritas: **A FAZER**, **FAZENDO** e **FEITO**. O sistema suporta dois papéis de usuário com privilégios diferenciados: **GESTOR** (responsável pela criação de projetos, quadros e cartões) e **DESENVOLVEDOR** (responsável pela execução e movimentação das atividades atribuídas).

O sistema adota o estilo de arquitetura em duas camadas: **Apresentação** e **Serviço**. A camada de apresentação realiza a interação com o usuário, valida os dados de entrada e solicita operações por meio de interfaces abstratas. A camada de serviço aplica as regras de negócio, verifica permissões, preserva os relacionamentos entre as entidades e utiliza contêineres repositórios para acessar um banco de dados relacional SQLite. A classe `BancoDados` gerencia a conexão compartilhada, cria as tabelas e os índices necessários e habilita a verificação das chaves estrangeiras. Os dados são mantidos no arquivo `kanban.db` entre execuções. As classes de domínio e entidade representam os dados utilizados pelas duas camadas.

2. Diagrama de Módulos, Interfaces e Relacionamentos (UML)

O diagrama a seguir especifica os módulos do sistema, as interfaces abstratas e as dependências entre os componentes. Os repositórios, `BancoDados` e o banco SQLite aparecem como componentes de apoio à camada de serviço.

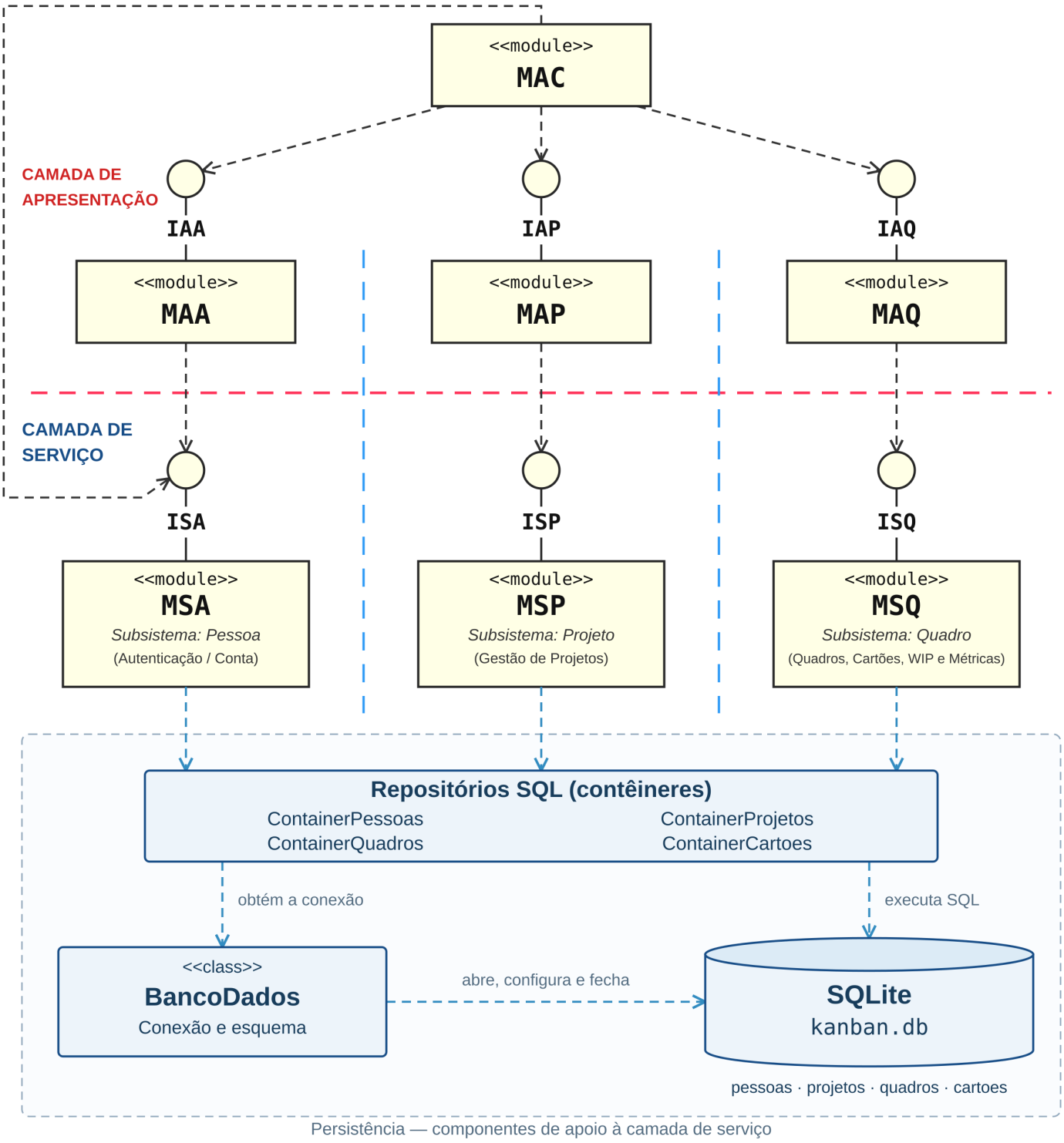


Figura 1 — Modelo de Arquitetura em Camadas com Interfaces Abstratas, Módulos Funcionais e Persistência SQLite.

Os repositórios obtêm de `BancoDados` a conexão compartilhada e utilizam a API do SQLite para executar os comandos SQL. Cada serviço utiliza os contêineres indicados nas seções 4.5 a 4.7.

3. Diagrama de Classes Entidade e Multiplicidades (UML)

O diagrama de entidades define as quatro classes fundamentais de dados do sistema, seus atributos tipados por classes de domínio, e os vínculos associativos e multiplicidades obrigatórias:

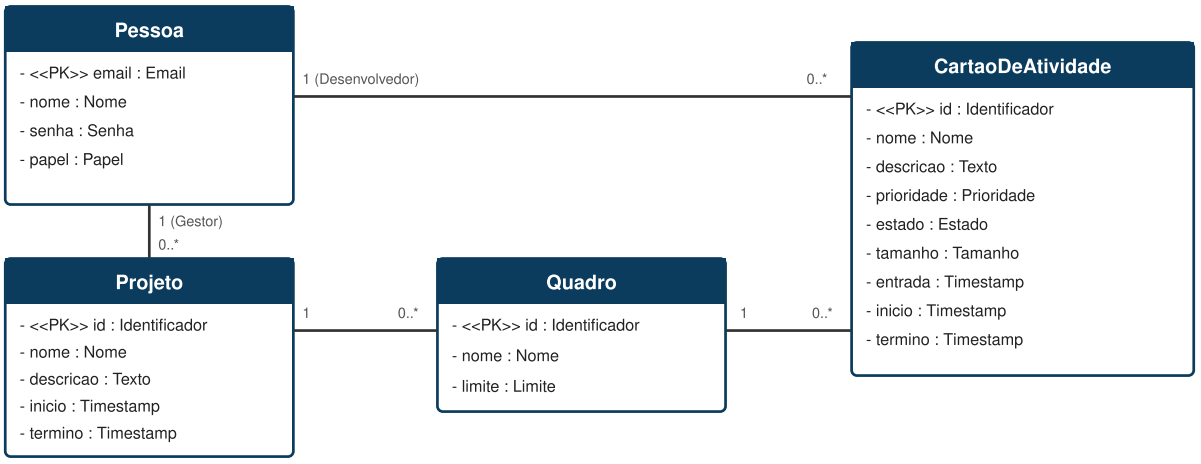


Figura 2 — Diagrama de Classes Entidade, Tipagem por Domínios e Multiplicidades.

4. Descrição Textual das Responsabilidades dos Módulos

A arquitetura possui sete módulos funcionais, distribuídos entre as camadas de apresentação e serviço, com o apoio dos repositórios e de `BancoDados` para a persistência SQLite. As siglas abaixo correspondem aos módulos e às interfaces identificados na Figura 1, que também representa os componentes de persistência.

4.1 MAC — Apresentação Principal

Implementado por `CntrApresentacaoPrincipal`, coordena o menu inicial, o acesso à sessão autenticada e a navegação entre os subsistemas. Solicita operações às interfaces de apresentação IAA, IAP e IAQ. Também depende de ISA (`IServicoPessoa`) para consultar os dados e o papel atual do usuário autenticado.

4.2 MAA — Apresentação de Pessoa

Implementado por `CntrApresentacaoPessoa`, fornece IAA (`IApresentacaoPessoa`). Realiza a interação relacionada ao cadastro, à autenticação e ao gerenciamento de pessoas. Coleta e valida os dados de entrada, solicita as operações por meio de ISA (`IServicoPessoa`) e apresenta seus resultados.

4.3 MAP — Apresentação de Projeto

Implementado por `CntrApresentacaoProjeto`, fornece IAP (`IApresentacaoProjeto`). Apresenta as operações de criação, leitura, atualização, exclusão e listagem de projetos, utilizando ISP (`IServicoProjeto`).

4.4 MAQ — Apresentação de Quadro

Implementado por `CntrApresentacaoQuadro`, fornece IAQ (`IApresentacaoQuadro`). Apresenta as operações de quadros e cartões de atividade, incluindo movimentação e consulta de métricas. Solicita essas operações por meio de ISQ (`IServicoQuadro`).

4.5 MSA — Serviço de Pessoa

Implementado por `CntrServicoPessoa`, fornece ISA (`IServicoPessoa`). Executa os serviços de cadastro, autenticação, leitura, atualização, exclusão e listagem de pessoas. Restringe a atualização e a exclusão de uma conta ao próprio usuário e verifica os vínculos com projetos e cartões antes da exclusão. Utiliza os contêineres de pessoas, projetos e cartões.

4.6 MSP — Serviço de Projeto

Implementado por `CntrServicoProjeto`, fornece ISP (`IServicoProjeto`). Executa as operações relacionadas aos projetos, verifica o papel e as permissões do usuário e mantém o vínculo entre projeto e gestor. Impede a exclusão de projetos que possuem quadros. Utiliza os

contêineres de pessoas, projetos e quadros.

4.7 MSQ — Serviço de Quadro

Implementado por `CntrServicoQuadro`, a qual cria e atualiza cartões, controla WIP e calcula tempo de ciclo e leade time. Fornece ISQ (`IServicoQuadro`), executa as operações relacionadas aos quadros, incluindo sua listagem por projeto. Verifica permissões, mantém o vínculo com o projeto e impede a exclusão de quadros que possuem cartões. Também impede a redução do limite abaixo do WIP atual. Utiliza os contêineres de pessoas, projetos, quadros e cartões.

4.8 Interfaces e Classes de Apoio

As seis interfaces abstratas são declaradas em `Headers/interfaces.hpp`. As interfaces de apresentação definem os contratos utilizados por MAC para acessar os demais módulos de apresentação. As interfaces de serviço definem os contratos utilizados para solicitar operações à camada de serviço.

As classes `ContainerPessoas`, `ContainerProjetos`, `ContainerQuadros` e `ContainerCartoes` implementam os repositórios SQL utilizados pelos módulos de serviço. Executam comandos preparados, com parâmetros vinculados, para cadastrar, consultar, atualizar e excluir registros. As chaves primárias garantem a unicidade dos identificadores, e as chaves estrangeiras preservam os vínculos entre as tabelas. As instâncias dos contêineres são criadas em `Main/main.cpp`, recebem por referência o mesmo objeto `BancoDados` e são compartilhadas por referência entre as controladoras de serviço.

As entidades Pessoa, Projeto, Quadro e CartaoDeAtividade representam os dados do sistema e possuem atributos compostos por classes de domínio. Os domínios validam formatos e valores por meio de seus métodos de atribuição, lançando `std::invalid_argument` quando recebem valores inválidos.

4.9 Persistência e Banco de Dados

A classe `BancoDados` é declarada em `Headers/banco_dados.hpp` e implementada em `BancoDados/banco_dados.cpp`. Abre a conexão SQLite, habilita a verificação das chaves estrangeiras e cria, em uma transação, as tabelas `pessoas`, `projetos`, `quadros` e `cartoes`, além dos índices necessários, quando ainda não existirem. A inicialização preserva os registros já armazenados. O destrutor fecha a conexão.

As chaves estrangeiras vinculam os projetos aos gestores, os quadros aos projetos e os cartões aos quadros e desenvolvedores. As restrições `ON DELETE RESTRICT` e `ON UPDATE RESTRICT` impedem excluir ou alterar chaves ainda referenciadas. As controladoras continuam responsáveis pelas permissões, pelos papéis dos usuários, pelo controle de WIP, pelas movimentações e pelos cálculos das métricas.

Em `Main/main.cpp`, `BancoDados` é criado antes dos contêineres e permanece válido durante o uso deles. O arquivo `kanban.db` é relativo ao diretório de execução; ao fechar e reabrir o programa nesse mesmo diretório, os registros permanecem disponíveis. O teste de fumaça das controladoras utiliza um banco SQLite isolado em memória (`:memory:`), descartado ao encerrar o teste, preservando o banco de uso normal.

5. Tabela de Mapeamento dos 26 Serviços Funcionais

ID	Serviço	Entidade	Papéis com Direito	Interface / Método Declarado
1	CRIAR	PESSOA	GESTOR, DESENVOLVEDOR	`IServicoPessoa::criar`
2	LER	PESSOA	GESTOR, DESENVOLVEDOR	`IServicoPessoa::ler`
3	ATUALIZAR	PESSOA	GESTOR, DESENVOLVEDOR	`IServicoPessoa::atualizar`
4	EXCLUIR	PESSOA	GESTOR, DESENVOLVEDOR	`IServicoPessoa::excluir`
5	LISTAR	PESSOA	GESTOR, DESENVOLVEDOR	`IServicoPessoa::listar`
6	CRIAR	PROJETO	GESTOR	`IServicoProjeto::criar`
7	LER	PROJETO	GESTOR, DESENVOLVEDOR	`IServicoProjeto::ler`
8	ATUALIZAR	PROJETO	GESTOR	`IServicoProjeto::atualizar`
9	EXCLUIR	PROJETO	GESTOR	`IServicoProjeto::excluir`
10	LISTAR	PROJETO	GESTOR, DESENVOLVEDOR	`IServicoProjeto::listar`
11	CRIAR	QUADRO	GESTOR	`IServicoQuadro::criar`
12	LER	QUADRO	GESTOR, DESENVOLVEDOR	`IServicoQuadro::ler`
13	ATUALIZAR	QUADRO	GESTOR	`IServicoQuadro::atualizar`
14	EXCLUIR	QUADRO	GESTOR	`IServicoQuadro::excluir`
15	LISTAR	QUADRO	GESTOR, DESENVOLVEDOR	`IServicoQuadro::listar`
16	CRIAR	CARTÃO DE ATIVIDADE	GESTOR	`IServicoQuadro::criarCartao`
17	LER	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::lerCartao`
18	ATUALIZAR	CARTÃO DE ATIVIDADE	GESTOR	`IServicoQuadro::atualizarCartao`
19	EXCLUIR	CARTÃO DE ATIVIDADE	GESTOR	`IServicoQuadro::excluirCartao`
20	LISTAR	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::listarCartoes`
21	MOVER	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::moverCartao`
22	LER TEMPO DE CICLO	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::lerTempoCiclo`
23	LER TEMPO DE CICLO MÉDIO	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::lerTempoCicloMedio`
24	LER LEAD TIME	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::lerLeadTime`
25	LER LEAD TIME MÉDIO	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::lerLeadTimeMedio`
26	LER WIP	CARTÃO DE ATIVIDADE	GESTOR, DESENVOLVEDOR	`IServicoQuadro::lerWIP`